

Data Encryption Standard

Historie

Data Encryption Standard (DES) je v kryptografii symetrická bloková šifra vyvinutá v 70. letech. V roce 1977 byla zvolena za standard pro šifrování dat v civilních státních organizacích v USA a následně se rozšířila i do soukromého sektoru. Do roku 1998 byl (s výhradami) použit jako standard pro šifrování informací. V současnosti je tato šifra považována za nespolehlivou, protože používá klíč pouze o délce 64 bitů, z toho 8 je kontrolních a 56 efektivních. Navíc obsahuje algoritmus slabiny, které dále snižují bezpečnost šifry. Díky tomu je možné šifru prolomit útokem hrubou silou řádově v jednotkách hodin.

Algoritmus DES je zástupcem symetrických blokových šifer. Symetrický znamená, že pro kódování i dekódování se používá stejný klíč a přívlastek blokový naznačuje, že šifruje data po blocích.

DES je založen na algoritmu firmy IBM, který se jmenoval Lucifer. Tento původní algoritmus byl proprietární a používal klíč délky 128 bitů. Při implementaci DESu došlo ke zkrácení klíče na pouhých 56 bitů. Toto zkrácení bylo vyvoláno bezpečnostním úřadem NSA kvůli obavám, že by algoritmus byl velmi silný a mohl by být zneužit pro nelegální činnost.

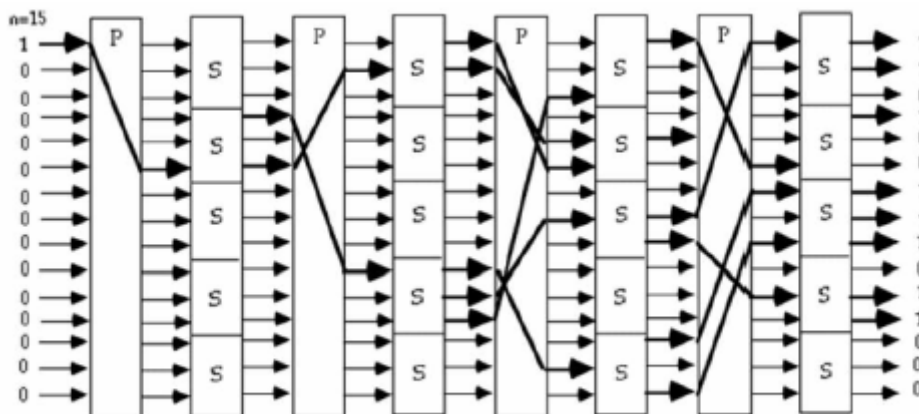
Zvýšení zabezpečení

Možným způsobem jak zvýšit bezpečnost této šifry, je vícenásobná aplikace. Tak vznikl algoritmus Triple DES, který je trojnásobnou aplikací šifry DES. Nejčastěji používaná varianta 3TDES pracuje s klíčem o celkové délce 168 bitů.

Triple DES je oproti novějším algoritmům (AES) daleko pomalejší, a proto se postupně přestal používat.

Základní vlastnosti

Lavinovitost (Avalanche): Jedná se o tzv. "Lavinovitý efekt". V praxi jde o to, že změna jednoho vstupního bitu má vliv na přibližně polovinu bitů na výstupu. Na následujícím obrázku je právě ukázka toho, jak může jeden bit na vstupu změnit mnoho výstupních bitů. Blok, který je na obrázku označen jako P je P-box a označení S znamená S-box. Vysvětlení těchto pojmů je níže.



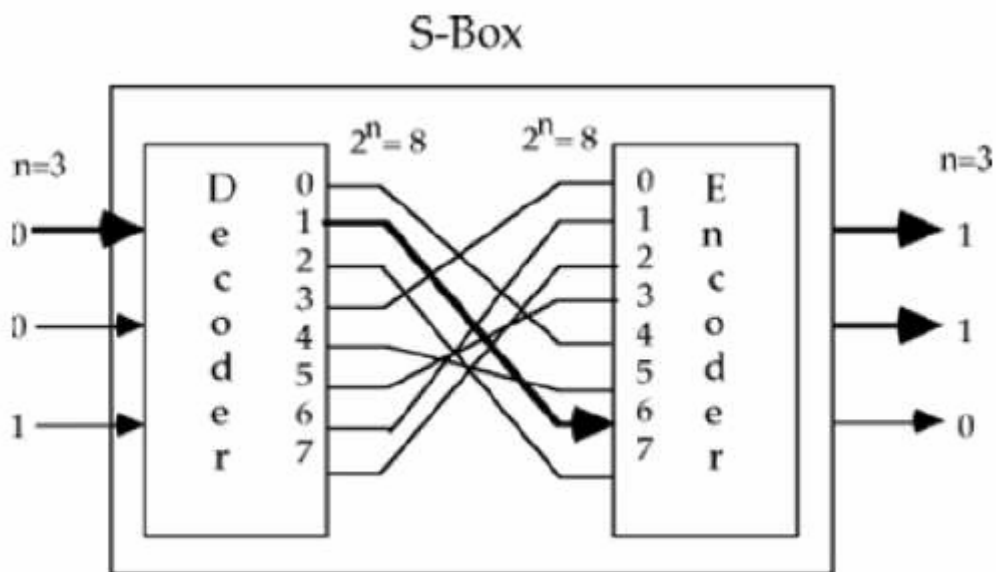
Úplnost (Completeness): Splnění této vlastnosti nám říká, že každý výstupní bit je určitou funkcí všech vstupních bitů. Výsledkem je komplikovaný vznik šifrovaného textu, který není jednoduché rozluštit. Na splnění této vlastnosti se u algoritmu DES nejvíce podílí nelinearita tzv. S-boxů, čili části algoritmu, která provádí substituci vstupních bitů na výstupní. Podrobněji bude popsáno dále. Pokud by zmiňované S-boxy byly lineární, pak by všechny bity šifrovaného textu byly jen lineární kombinací nezašifrované zprávy a klíče. V tomto případě by stačila soustava lineárních rovnic k získání nezašifrované zprávy.

Neexistence korelace: Dále bylo statisticky ověřeno, že neexistuje korelace (vztah) mezi otevřeným textem (nezašifrovanou zprávou) a šifrovaným textem a dále, že také neexistuje korelace mezi šifrovaným textem a klíčem. To znamená, že není možné nalézt vztahy mezi otevřeným textem, klíčem a šifrou. Tím je prokázána vyšší bezpečnost algoritmu. Pokud by tomu bylo naopak, potom by stačilo jednou nalézt vztahy mezi klíčem, šifrou a zprávou. Po zveřejnění těchto vztahů by prolomení šifry (získání otevřeného textu ze šifrovaného) bylo velmi jednoduché.

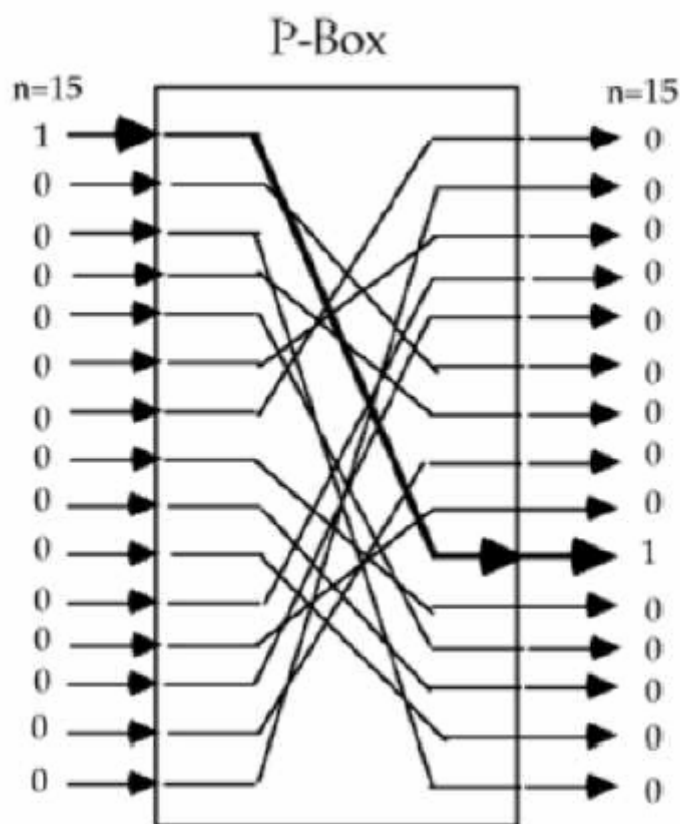
Stavební bloky

Jelikož DES je složeným algoritmem, je nutné si nejdříve představit základní stavební bloky, se kterými se pracuje. Tyto bloky je možné si například představit jako funkce v programu.

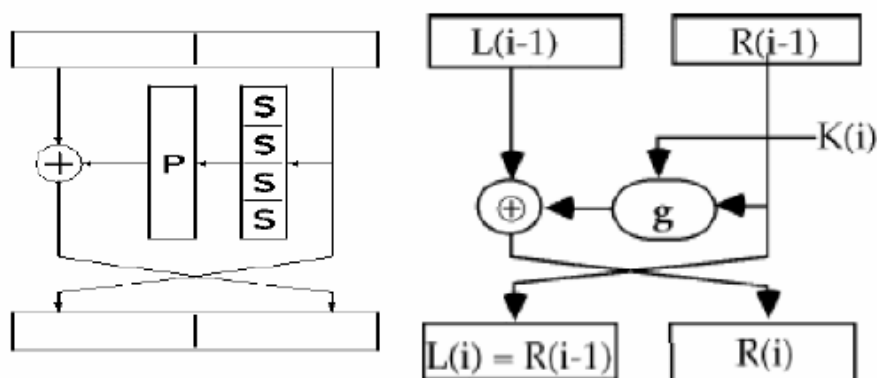
S-Box: Jedná se o část algoritmu, která provádí substituci, čili zaměnění bitů, které jsou na vstupu tak, že na vstup dá jinou kombinaci bitů. Jak má funkce pracovat a jaké bity má dávat na výstup v závislosti na vstupu je určeno uvnitř této funkce. Na dalším obrázku je uveden příklad. Je vidět, že bity na vstupu jsou podle určitého předpisu zaměněny a dány na výstup. Výhodou S-Boxu je to, že nemusí být nutně shodný počet vstupních a výstupních bitů. Tohoto se využívá právě v algoritmu DES.



P-Box: Tato část má obdobnou funkci jako S-Box. Je ale jednodušší, protože provádí pouze permutaci bitů. To znamená, že pouze "přehází pořadí" jednotlivých vstupních bitů. Na obrázku je opět uveden příklad.



Feistelova síť: Jedná se o znázornění funkcí šifrovacího algoritmu. Popisuje, jak se provádí jeden krok šifrování i to, kolikrát se jednotlivé kroky provádějí. Jeden krok potom probíhá následovně: Blok zprávy se rozdělí na dvě poloviny (označme levou a pravou polovinu). Z klíče si vytvoříme podklíč pro tento krok a provedeme vlastní šifrování. Jako levou polovinu výstupu dáme pravou polovinu vstupu (čili polovina vstupních bitů se nemění, pouze se prohodí). Pravou polovinu výstupu dostaneme jako kombinaci levé poloviny vstupu, změněné pravé poloviny (pomocí S a P-Boxů) a podklíče. Pro lepší pochopení si uvedeme popis pomocí obrázku.



Blok zprávy, který šifrujeme se rozdělí na dvě poloviny

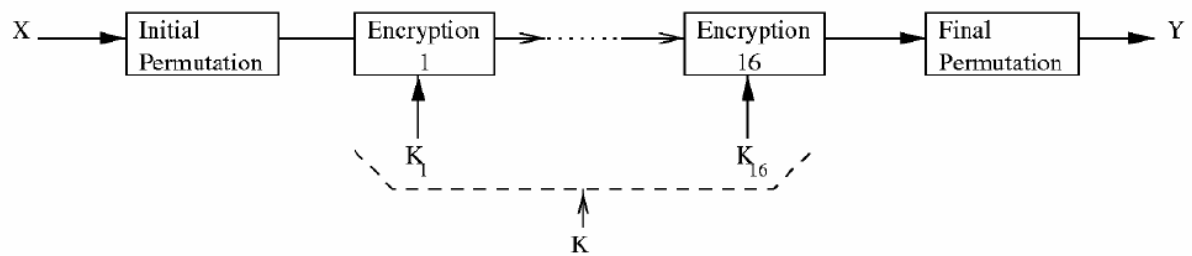
- Pravou polovinu vstupního bloku zkopírujeme jako novou levou polovinu výstupu
- Pravou polovinu vstupu změním pomocí S a P Boxů (zde může vstoupit do hry klíč)
- Změněnou pravou polovinu sloučíme s levou polovinou vstupu pomocí funkce XOR
- Výsledek po spojení zapíšeme jako pravou polovinu výstupu

Vlastní průběh šifrování

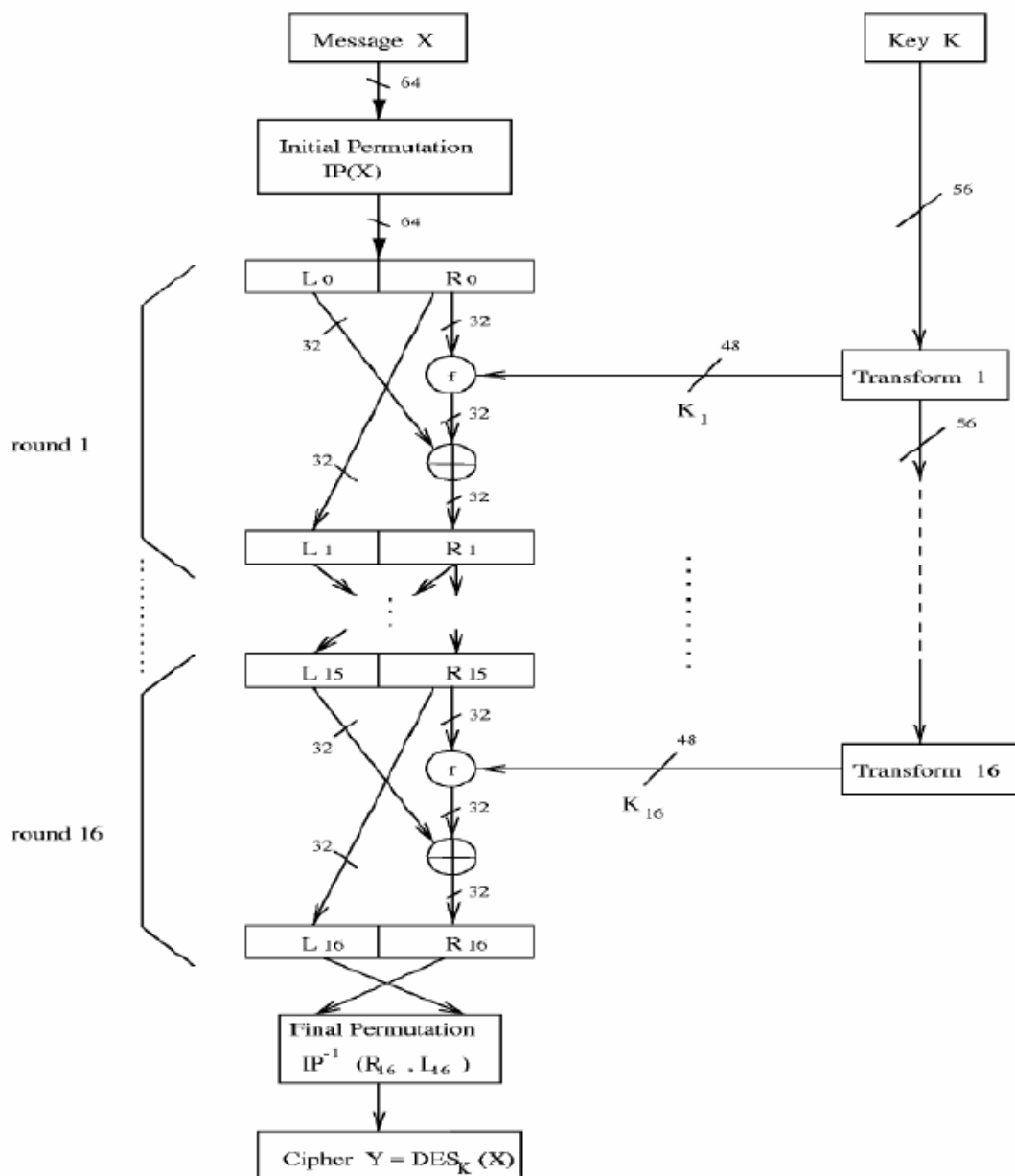
DES pracuje s bloky dat o délce 64 bitů. To znamená, že zpráva je rozdělena na bloky o velikosti právě 64 bitů a s těmito je dále pracováno. Délka klíče je 56 bitů, ale klíč se nejčastěji uvádí také jako 64 bitová hodnota. V tomto vyjádření ale slouží každý osmý bit jako paritní (bezpečnostní) a při vlastním šifrování se ignoruje. Přesněji každý nejnižší bit (bit s nejnižší váhou) slouží jako lichá parita. Dostáváme tak opět klíč o délce 56 bitů. Při šifrování se provádí několik kroků. Prvním z nich je úvodní permutace bloku zprávy. Ta se provádí podle následující tabulky a to tak, že vstupní osmibitová hodnota slouží jako index do této tabulky a na výstup se dává hodnota nalezená v tabulce. 4 bity jsou indexem řádku a zbylé 4 bity jsou indexem sloupce. Na výstup se poté vloží hodnota, která se nachází na průsečíku řádku a sloupce.

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

Bylo prokázáno, že znalost této počáteční permutace nemá žádný vliv na bezpečnost algoritmu. Po provedení následuje 16 kroků šifrování. Průběh jednoho kroku byl popsán výše u Feistelovy sítě, dále si ho popíšeme přesně tak, jak se provádí v DESu. V této fázi přichází ke slovu i klíč, ze kterého se generuje 16 podklíčů (pro každý krok jiný) a tímto klíčem je ovlivněn výsledek šifrování. Posledním krokem je provedení konečné permutace a výsledkem je zašifrovaný blok zprávy podle klíče, který byl použit. Na dalším obrázku je znázorněn obecný průběh šifrování.



Závěrečný obrázek ukazuje celkové schéma průběhu včetně zapojení klíče do šifrování:



Na konci algoritmu následuje už jen výstupní permutace, která je inverzní k vstupní (první) permutaci a výsledkem je zašifrovaný blok zprávy.

Dešifrování zprávy

Velkou výhodou je, že pro dešifrování zprávy se pouze otočí pořadí kroků při šifrování. Může tedy být využito stejných funkcí, které se používají při šifrování. Jen je nutné postupovat "od konce", čili začít vždy od 16té rundy a pokračovat k první. To se týká hlavně generování podklíčů, kdy také dochází k bitovým posunům, ale doprava.

Výhodou tohoto řešení je, že může být použit stejný hardware pro obě operace. To umožňuje vyrábět hardwarové prostředky implementující DES, které mohou využívat pouze nezbytně nutné prostředky. Kdybychom porovnali jejich složitost s případem, kdy by se pro dešifrování použil úplně jiný postup, tak použité řešení přináší přibližně poloviční složitost buď programu nebo hardwarových částí.

Nutné vlastnosti použitých klíčů

Protože je s klíči nakládáno tak, jak je popsáno, vzniká omezení pro použití možných klíčů. Obecně je možné použít jakoukoli 56 (resp. 64) bitovou hodnotu. Některé z nich ale snižují bezpečnost výsledného šifrování. Říkáme o nich potom, že jsou to klíče slabé nebo poloslabe. Příklady některých z nich jsou uvedeny v následujících tabulkách.

Slabý klíč (hexa)	C0	D0
0101 0101 0101 0101	$\{0\}^{28}$	$\{0\}^{28}$
FEFE FEFE FEFE FEFE	$\{1\}^{28}$	$\{1\}^{28}$
1F1F 1F1F 1F1F 1F1F	$\{0\}^{28}$	$\{1\}^{28}$
E0E0 E0E0 E0E0 E0E0	$\{1\}^{28}$	$\{0\}^{28}$

C0	D0	Pár poloslabyých klíčů (hexa)	C0	D0
$\{01\}^{14}$	$\{01\}^{14}$	01FE 01FE 01FE 01FE, FE01 FE01 FE01 FE01	$\{10\}^{14}$	$\{10\}^{14}$
$\{01\}^{14}$	$\{10\}^{14}$	1FE0 1FE0 0EF1 0EF1, E01F E01F F10E F10E	$\{10\}^{14}$	$\{01\}^{14}$
$\{01\}^{14}$	$\{0\}^{28}$	01E0 01E0 01F1 01F1, E001 E001 F101 F101	$\{10\}^{14}$	$\{0\}^{28}$
$\{01\}^{14}$	$\{1\}^{28}$	1FFE 1FFE 0EFE 0EFE, FE1F FE1F FE0E FE0E	$\{10\}^{14}$	$\{1\}^{28}$
$\{0\}^{28}$	$\{01\}^{14}$	011F 011F 010E 010E, 1F01 1F01 0E01 0E01	$\{0\}^{28}$	$\{10\}^{14}$
$\{1\}^{28}$	$\{01\}^{14}$	E0FE E0FE F1FE F1FE, FEE0 FEE0 FEF1 FEF1	$\{1\}^{28}$	$\{10\}^{14}$

V tabulce jsou příklady slabých klíčů, kdy po zašifrování dostaneme stejný text, jako před šifrováním. Nedojde tedy k vůbec žádné změně ve zprávě a ta je čitelná i po zašifrování. Druhá tabulka obsahuje příklady poloslabyých klíčů. V tomto případě jde o to, že pokud zprávu zašifrujeme nejdříve prvním z uvedeného páru klíčů a poté druhým, tak také dostaneme stejný text, jako na začátku (nezměněnou zprávu). Oběma typům klíče je tedy nutné se vyhnout.

Zdroj, podrobnosti k algoritmu DES: <http://homel.vsb.cz/~mor196/mal338.pdf>

Advanced Encryption Standard

AES je schválený standard amerického úřadu pro standardizaci (NIST), který byl udělen symetrické blokové šifře Rijndael. Šifra byla vyvinuta Belgičany Joanem Daemenem a Vincentem Rijmenem a zařazena do veřejné soutěže NIST vyhlášené 2. 1. 1997 o nový federální šifrovací systém.

Šifra využívá symetrického klíče. Tj. stejný klíč je použit pro šifrování i dešifrování. Délka klíče může být 128, 192 nebo 256 bitů. Metoda šifruje data postupně v blocích s pevnou délkou 128 bitů. Šifra se vyznačuje vysokou rychlostí šifrování.

V současné době není veřejně znám žádný případ plného prolomení této metody ochrany dat. AES je nástupcem zastaralého šifrovacího standardu DES, který byl 17.6.1997 prolomen.

Další z pěti finalistů druhého kola na AES jsou algoritmy MARS, RC6, Serpent a Twofish, které v soutěži vybrány nebyly, nicméně není ojedinělé, že jsou jako doplněk k AES implementovány do šifrovacích programů (např. open source FreeOTFE a TrueCrypt.)

Délka klíče

AES vychází z 25 let starých principů, které byly použity i pro DES. Velký pokrok oproti DES je délka klíčů. AES podporuje délky klíčů 128, 192 a 256 bitů. Bohužel ani AES není nerozluštitelná a určitě se dříve nebo později objeví něco jako DES Cracker v podání pro AES, v současné době však není veřejně známý obecný útok na šifru AES.

Šifrování

Rijndael je blokový šifrovací algoritmus. Je aplikován na data s pevně danou délkou - v tomto konkrétním případě 128 bitů. Pokud jsou šifrovaná data delší, zpracovávají se po jednotlivých blocích. Pokud jsou data kratší (typicky v případě posledního bloku se zbytkem dat), je potřeba je doplnit na odpovídající délku.

Onomu doplnění na odpovídající délku se říká "padding". Existuje pro něj několik algoritmů, od primitivního doplnění nulami po složitější schémata (podrobnější popis najdete např. v RFC2315).

Naivní implementace blokové šifry by spočívala v tom, že by se šifra aplikovala na jednotlivé bloky tak, jak jdou za sebou. Tomuto postupu se říká **ECB (Electronic Codebook)** a jeho použití se nedoporučuje. Jeho výsledkem totiž je, že stejné bloky otevřeného textu budou zašifrovány vždy stejně. Vzhledem k poměrně malé velikosti jednotlivých bloků to může znamenat, že ve výsledné zprávě budou patrná stejná schémata jako v původním dokumentu.

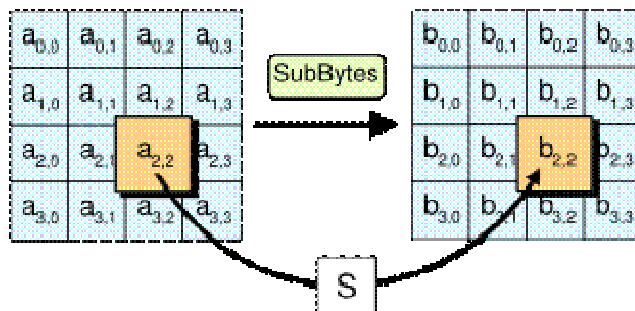
V praxi se tedy používají takové algoritmy, které toto nebezpečí odstraňují. Nejjednodušším z nich je **CBC - Cipher Block Chaining**. Ten funguje tak, že před zašifrováním se odpovídající blok otevřeného textu XORuje předcházejícím blokem zašifrovaného textu. To znamená, že jednotlivé bloky jsou na sobě závislé, abyste dešifrovali konkrétní blok, musíte dešifrovat i všechny předchozí.

V tomto případě ale máte problém - čím XORovat první blok, když ještě nemáte k dispozici blok předchozí? Inu, použijeme osvědčenou metodu socialistických vlád: když informace nemáme, prostě si je vymyslíme. Jeden blok se vygeneruje náhodně a přidá se k zašifrovaným datům jako "nulový blok". Tomuto bloku se pak říká "inicializační vektor" (IV). Tento blok se použije k dešifrování prvního bloku a pak zahodí.

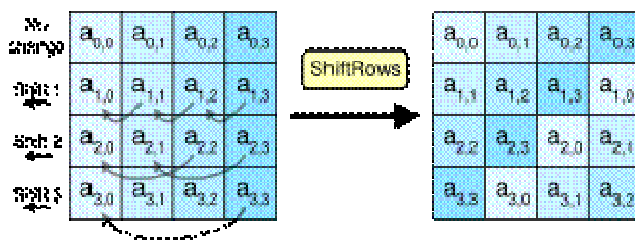
Popis šifrování

Šifrování probíhá ve čtyřech krocích:

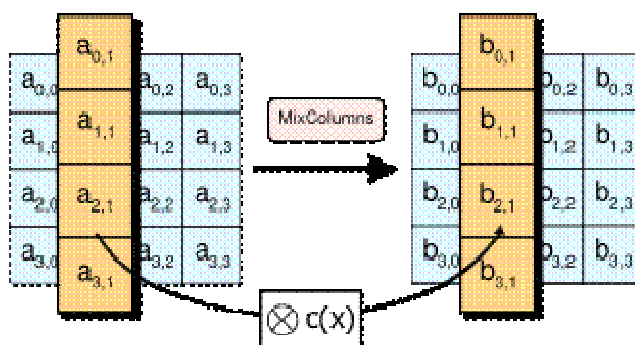
SubBytes - jednoduchá substituce, kde každý bajt je nahrazen jiným podle předem daného klíče, tzv. Rijndael-S-Box (8bitů). Tato operace zajišťuje nelineárnost šifry a má zabránit útokům založeným na jednoduchých algebraických vlastnostech.



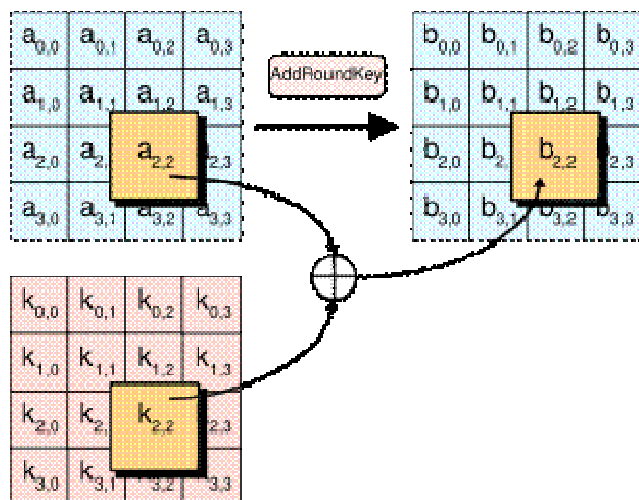
ShiftRows — v tomto kroku se jednotlivé byty přehází podle obrázku:



MixColumns — při této operaci dochází k proházení sloupců 4 bajtů a zároveň je každý sloupec vynásoben stejným polynomm $c(x)=8x^3+x^2+x+2$



AddRoundKey — každý byt je zkombinovaný se subklíčem (subklíč získáme z původního klíče pomocí Rijndaelovy tabulky). Každý byt subklíče zkombinujeme s příslušným bytem naší zprávy a dostaneme výslednou šifru



Dešifrování

Při **dešifrování** probíhají inverze jednotlivých operací v obráceném pořadí než při šifrování:

AddRoundKey — operace XOR je sama o sobě invertibilní

MixColumn – inverzi realizujeme pomocí násobení inverzním polynomem

ShiftRows – inverze se realizuje jako bitový posun doleva

SubBytes – jediná nelineární operace se vrátí invertuje pomocí vyhledávání v tabulce

Útoky na AES

V roce 2002 byl představen teoretický útok proti AES s názvem eXtended Sparse Linearization. V roce 2005 byl podán důkaz, že XSL není efektivní a že jeho režie je tak velká jako u útoku hrubou silou.

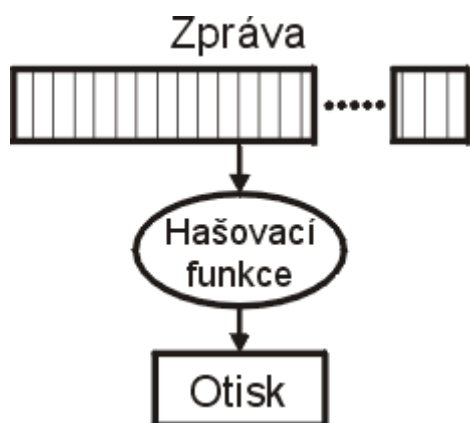
Od roku 2006 jsou známy toky na implementaci AES pomocí postranních kanálů. Pomocí postranních kanálů se neutočí na samotný algoritmus, ale implementaci – kryptoanalytik musí mít k dispozici otevřený text zašifrovaný mnoha klíči, které splňují určitá pravidla.

Zdroj: http://en.wikipedia.org/wiki/Advanced_Encryption_Standard

Úvod do hašovacích funkcí

Hašovací funkce je funkce h , která má přinejmenším tyto vlastnosti: je kompresní - provádí mapování argumentu /vstupu/ x libovolné bitové délky na hodnotu $h(x)$ /výstup/, která má pevně určenou bitovou délku, je snadno vypočitatelná – pro dané h a argument x je snadné vypočítat $h(x)$.

Tuto vlastnost ilustruje obrázek č.1, na němž je argument označen jako „zpráva“ a hodnota funkce slovem „otisk“. Smyslem použití hašovacích funkcí obecně bývá možnost reprezentovat potenciálně i velmi dlouhou zprávu jen jejím krátkým otiskem a vytvořit pomyslnou relaci 1:1 mezi zprávou a otiskem.



Kompresní vlastnost všech hašovacích funkcí

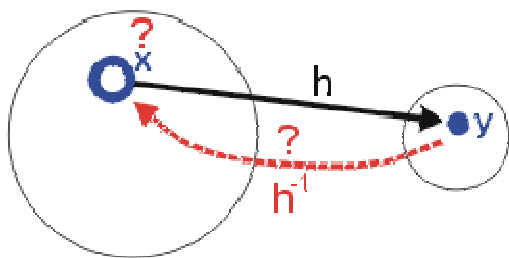
Hašovací funkce mají v počítačové praxi velmi mnoho použití. V diagnostice hardware např. indikují konečné stavy testů, v komunikaci se používají pro detekci chyb (kódy CRC apod.), v softwarových technikách např. pro implementaci tabulek atd. Ve všech těchto případech se zpravidla vyžaduje, aby hašovací funkce prováděla i dobrou náhodnou distribuci výsledků, rovnoměrné pokrytí oboru hodnot, aby malá změna vstupního řetězce vedla jistě k odlišnému výsledku (ideálně lavinově jinému).

Hašovací funkce vyhovující výše uvedeným aplikacím sice dobře detekují „náhodné“ změny argumentů (zprávy), nejsou však dostatečné pro obor kryptologie, ve kterém lze předpokládat, že útočník bude postupovat při změnách argumentů funkce s veškerou možnou inteligencí.

Kryptologové se proto, mimo jiné, zabývají hašovacími funkcemi (funkci značíme h , argumenty x a x' /vstupy/, hodnoty y a y' /výstupy/), které mají následné tři vlastnosti:

Odolnost argumentu (preimage resistance) – pro v podstatě všechny výstupy je výpočtově neschůdné nalézt jakýkoliv vstup, který hašuje na daný výstup; tj. nalézt jakýkoliv argument x takový, že $h(x) = y$ pro danou hodnotu y , pro niž odpovídající vstup není předem znám.

Pro ne-matematiky ilustruje tuto vlastnost další obrázek..



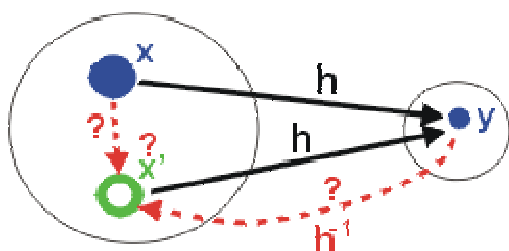
Odolnost argumentu (též „jednocestnost“)

Tato vlastnost polopaticky řečeno znamená, že z otisku nesmíme být schopni zjistit jakoukoliv zprávu, které by odpovídal.

Předem známa je pouze funkce h , tj. způsob jejího výpočtu, definiční obor (kruh vlevo) i obor hodnot (kruh vpravo) této funkce. Z oboru hodnot je volen libovolný výstup y . Nalezení jakéhokoliv vstupu x funkce h , který by mu odpovídal, nesmí být možné. Definice uvedená výše připouští, že útočník může předpočítat obraz (hodnoty y) pro nějakou zanedbatelně malou podmnožinu vzorů z definičního oboru – takové způsoby nalezení argumentu se neuvažují, protože se předpokládá, že se útočník na y netrefí.

V praxi je v takové situaci například útočník na heslo x , které je uloženo formou zhašované hodnoty y v souboru na disku, k němuž se mu podařilo získat přístup (pro ztížení slovníkových útoků bývají hodnoty hesla ještě tzv. osoleny, ale to zde rozebírat nebudeme).

Odolnost druhého argumentu (2nd-preimage resistance) – je výpočtově neschůdné nalézt jakýkoliv druhý vstup, který má shodný výstup jako jakýkoliv určený první vstup; tj. pro dané x nelze nalézt druhý argument $x' \neq x$, přičemž platí $h(x) = h(x') = y$.

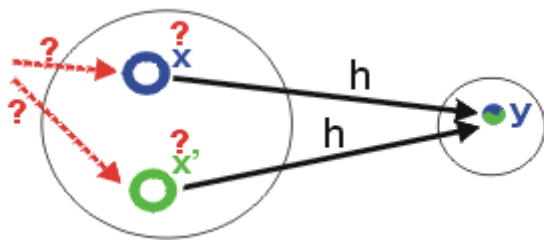


Odolnost druhého argumentu (též „slabá odolnost proti kolizím“)

Tato vlastnost se projevívá v „normální“ situaci, kterou si většina uživatelů představí při útoku na hašovací funkci. Máme předem danou hašovací funkci h , nějakou zprávu x a její otisk y (nebo ho snadno spočteme). Nesmí být možné nalézt jakoukoliv jinou zprávu x' , jejíž otisk by byl stejný. Protože se předem neví, jakou zprávu budeme hašovat, výše uvedená definice požaduje tuto vlastnost pro všechny argumenty x .

V praxi přichází do úvahy útok tehdy, pokud by chtěl někdo zaměnit zprávu x za x' , což může přicházet do úvahy např. při (elektronickém) digitálním podpisu, při kontrole integrity souborů (ať již stahovaných z webu nebo P2P sítí). Běžný člověk by si pravil, že to by mohlo stačit, kryptologové však vznášejí ještě třetí požadavek.

Odolnost proti kolizím (collision resistance) – je výpočtově neschůdné nalézt jakékoliv dva rozdílné vstupy x a x' , které hašují na shodný výstup; tj. $x' \neq x$ takové, že $h(x) = h(x') = y$ (volný výběr obou vstupů).



Odolnost proti kolizím (též „silná odolnost proti kolizím“)

Tato situace se liší od předchozí tím, že si je možné hodnotu x volit libovolně. Na první pohled se zdá, že se jedná o shodné definiční podmínky jako v předchozím případě, ale rozhodně tomu tak není. Danost zprávy y (prvního argumentu) je totiž při zkoušení odolnosti druhého argumentu určena nějakou, na útočníkovi nezávislou, podmínkou. Z pohledu útočníka je x již dáno a musí pro něj hledat x' .

Při obecné (silné) odolnosti proti kolizím si však útočník může volit x i x' s jediným cílem, totiž aby hašovaly na shodnou hodnotu (lhostejno jakou). Tato vlastnost předpokládá, že útočník je v takové pozici, že může ovlivnit vstupní hodnotu hašovací funkce. Další viz níže u útoku Wangové, které se všechny zaměřují na toto obecné hledání kolizí (dle obr.4).

Ideální kryptografická hašovací funkce bude oplývat všemi třemi vlastnostmi! Je vhodné chápat, že pokud je velikost množiny definičního oboru hašovací funkce větší než oboru hodnot (což bývá v praxi vždy), hašovací funkce nemůže být bezkolizní.

Pozorný čtenář si však jistě všiml, že definice obsahují obraty „výpočtově neschůdné“ - kolize prostě nesmí být možné nalézt. Kontext definice se mění jak rozvojem technologie, tak požadovanou mírou ekonomických nákladů (nemá smysl utrácet více, než získáte).

Narozeninový paradox

Zajímavé rovněž je, že obě zmíněné situace (odolnost druhého argumentu a odolnost proti kolizím) mají rozdílný výpočet pravděpodobností náhodného nalezení kolizí. Druhá situace je v kryptologii známa jako tzv. narozeninový paradox.

Jste-li na večírku, pak pro 100% pravděpodobnost toho, že se na něm nachází někdo, kdo se narodil shodný den v roce (léta se neuvažují) jako právě vy, je zapotřebí, aby tam přišlo alespoň 366 osob (musíte jít na megapárty). Pokud se však spokojíte s 50procentní pravděpodobností toho, že se na večírku nachází libovolné dvě osoby se shodným narozeninami, pak dostačuje, aby večírek navštívilo 23 osob.

Pozn: narozeninový paradox samozřejmě předpokládá náhodnost narozenin dotyčných, tj. že nejste třeba na vítání občánků, ani nejste dvojčata, chodící spolu kazit statistiky. Uvedené pravděpodobnosti též zanedbávají přestupné roky.

Tato nevinná matematická hříčka má v kryptologii vážný dopad. Útoky hrubou silou jsou v případech použitelnosti narozeninového paradoxu totiž mnohem snazší a říká se jim pak narozeninový útok.

Message-Digest algorithm

Message-Digest algorithm je rozšířená rodina hašovacích funkcí, která vytváří ze vstupních dat výstup (otisk) fixní délky. Otisk je též označován jako miniatura, kontrolní součet (v zásadě nesprávné označení), fingerprint, hash (česky někdy psán i jako haš).

Jeho hlavní vlastností je, že malá změna na vstupu vede k velké změně na výstupu, tj. k vytvoření zásadně odlišného otisku.

MD5

Algoritmus MD5 se prosadil do mnoha aplikací (např. pro kontrolu integrity souborů nebo ukládání hesel). MD5 je popsán v internetovém standardu RFC 1321 a vytváří otisk o velikosti 128 bitů. Byl vytvořen v roce 1991 Ronaldem Rivestem, aby nahradil dřívější hašovací funkci MD4.

Bezpečnost

V roce 1996 byla objevena vada v návrhu MD5, a i když nebyla zásadní, kryptologové začali raději doporučovat jiné algoritmy, jako je například SHA (i když ani ten již dnes není považován za bezchybný). V roce 2004 byly nalezeny daleko větší chyby a od použití MD5 v bezpečnostních aplikacích se upouští.

Příklad kontrolního součtu MD5

Otisk 43 bajtového znakového řetězce (vyjádřený v hexadecimálním zápisu):

MD5("The quick brown fox jumps over the lazy dog") =

9e107d9d372bb6826bd81d3542a419d6

Stačí malá změna vstupního řetězce, aby byl otisk úplně odlišný (např. změňme d na c):

MD5("The quick brown fox jumps over the lazy cog") =

1055d3e698d289f2af8663725127bd4b

Zvýšení bezpečnosti

MD5 se často používá pro ukládání hesel. Přidáním soli k heslu se ztěžují útoky na získání hesla a účinnost slovníkových útoků s využitím předpočítaných tabulek (anglicky rainbow table attack). Tento postup lze použít i pro jakékoli kryptografické

hašovací funkce, které lze dopředu "předpočítat", jejich výsledný haš uložit a pak rychleji vyhledat v databázi srovnáváním s hašem, na který se útočí.

hash = MD5 (heslo . sul)

Pro jisté zvýšení bezpečnosti je možné kombinovat například heslo a uživatelské jméno, v takovém případě pokud dva uživatelé použijí totožné heslo, otisk (haš) jejich hesel bude zásadně odlišný, protože jejich uživatelská jména se určitě budou lišit. Další možností částečného zvýšení bezpečnosti je použití více hašovacích algoritmů najednou, například kombinace MD5 a SHA. Postup zajistí vyšší odolnost chráněné informace v případě, že bude při jedné z funkcí nalezena kolize. Například:

SHA1(MD5("login").MD5("heslo"))

Secure Hash Algorithm

SHA navrhla americká NSA a jako standard ho vydal NIST (Národní institut pro standardy v USA) jako americký federální standard. SHA je rodina pěti algoritmů: SHA-1, SHA-224, SHA-256, SHA-384 a SHA-512. Poslední čtyři varianty se souhrnně uvádějí jako SHA-2. SHA-1 vytvoří obraz zprávy dlouhý 160 bitů. Čísla u ostatních čtyř algoritmů značí délku výstupního otisku v bitech.

SHA-1 rozsekává vstupní zprávu na bloky o délce 512 bitů, poslední blok zprávy doplňuje a zarovnává, včetně přidání údaje o délce zprávy, na něž je vyhrazeno posledních 64 bitů. SHA-1 tak může zpracovávat vstupní zprávy o délce až do $2^{64}-1$ bitů, tj. cca 2.305.840 TB. Uvedením délky se výrazně ztěžuje možnost nalezení a výskytu kolizí mezi zprávami různých délek – kolize je primárně potřeba hledat mezi zprávami zcela shodné délky. Výstup SHA-1 má délku 160 bitů, tj. 20 bytů.

SHA-1 je, podobně jako jiné hašovací funkce, tzv. iterativní hašovací funkce. To znamená, že při zpracování každého 512bitového bloku se vždy použije stejný modul tzv. vnitřní kompresní funkce.

Kompresní funkce má dva vstupy, 160bitový (v prvním kroku je inicializován předepsaným inicializačním vektorem, v dalších krocích přenosy z výstupů minulých kroků jako tzv. kontext) a 512bitový (blok dat). Po zpracování posledního bloku se výstup kompresní funkce použije jako výstupní hodnota.

Použití

SHA se používá v mnoha různých protokolech či aplikacích, včetně TLS a SSL, PGP, SSH, S/MIME a IPsec, ale je využívána i pro kontrolu integrity souborů nebo ukládání hesel. SHA je považována za nástupce hašovací funkce MD5.

Útoky na MD5, SHA-1 a SHA-2

Pro prolomení prvního kritéria (tedy nalezení zprávy, která koresponduje se svým otiskem), může být použita hrubá síla při provedení 2^L výpočtů, kde L je velikost bitů v otisku zprávy. Toto se nazývá vzorový útok (preimage attack).

Útok na druhé kritérium (nalezení dvou rozdílných zpráv, které mají stejný otisk), které nazýváme kolize, vyžaduje jen $2^{L/2}$ výpočtů k provedení tzv. narozeninového útoku (birthday attack).

Útok na MD5 hash byl proveden čínským týmem profesorky Xiaoyun Wangové, který se jím v roce 2004 proslavil. Svou schopnost útoku na MD5 (a na další hašovací funkce) prokázal zveřejněním výsledných kolizí. Na počítači IBM P690 (32ti procesorový stroj) má nalezení kolize pro MD5 trvat asi hodinu. Jelikož si (alespoň někteří) kryptologové v rámci akademické spolupráce rádi navzájem dávají hádanky, Číňané podstatné podrobnosti své metody nezveřejnili. Ostatní kryptologové se následně během podzimu 2004 pokoušeli přijít ze zveřejněných dat a informací na to, jak to bylo provedeno.

Předního českého kryptologa Vlastimila Klímu pohltilo téma nadmíru, takže mu věnoval tři měsíce. Dne 5.března 2005 pak mezinárodně publikoval článek s provokativním názvem "Nalézání kolizí v MD5 – hračka pro notebook". Patrně ale nepřišel přesně na metodu Wangové, ale našel svou, která je celkově asi 4-6x rychlejší. Den poté, tj. 6. března, nechala prof. Wangová zveřejnit články detailně popisující prolomení MD5 a dalších funkcí. Tím se opět potvrdilo, že nejlepším způsobem uvolnění embarga na konkrétní informace je vytvoření vlastní náhrady.

Prof. Wangová také v únoru 2005 popsala útok na SHA-1 v řádu 2^{63} pokusů. Poté následovaly další práce, poslední byla prezentována na konferenci Eurocrypt 2009 od autorů Camerona McDonalda, Philipa Hawkese a Josefa Pieprzyka a ukazuje kolizi se složitostí v řádu 2^{52} výpočtů.

Budoucnost

V současnosti probíhají snahy o vývoj vylepšených hešovacích funkcí. NIST usiluje o zavedení jednoho nebo více dalších hešovacích algoritmů pomocí veřejné soutěže stejně jako u hledání Advanced Encryption Standard (AES) - první návrhy na SHA-3 byly podávány do 31. října 2008 a vyhlášení vítěze a publikace nového standardu je naplánováno na rok 2012.

Zdroje:

<http://www.lupa.cz/clanky/hasovaci-funkce-jak-se-odolava-hackerum/>
<http://www.lupa.cz/clanky/zustava-elektronicky-podpis-bezpecny/>
http://en.wikipedia.org/wiki/SHA_hash_functions
http://cs.wikipedia.org/wiki/Narozeninový_problém
http://cs.wikipedia.org/wiki/Secure_Hash_Algorithm